

LAB 1 : Empirical Analysis of Insertion sort

Theory:

Insertion Sort is a simple and intuitive sorting algorithm that builds the final sorted array one item at a time, much like sorting a hand of playing cards. It works by taking an un-sorted element, comparing it to the items before it, and "inserting" it into its correct sequential position.

Because it iterates through the list and shifts elements over to make room for the current item, its performance is highly dependent on how sorted the initial data is. If the array is already mostly sorted, it does very little work ($O(n)$). However, if the array is completely random or in reverse order, it has to shift virtually every element on every pass, resulting in a much slower $O(n^2)$ performance.

```
void insertionSort(int array[], int size) {  
  
    for (int step = 1; step < size; step++) {  
  
        int key = array[step];  
  
        int j = step - 1;  
  
  
        while (j >= 0 && array[j] > key) {  
  
            array[j + 1] = array[j];  
  
            j--;  
  
        }  
  
        array[j + 1] = key;  
  
    }  
}
```

Expressing complexity of cases:

- **Best Case:** $T(n) = O(n)$ (*Array is already sorted*)
- **Worst Case:** $T(n) = O(n^2)$ (*Array is reverse sorted*)

Source Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

long long count = 0;

void generateAscendingArray(int *arr, int size) {

    for (int i = 0; i < size; i++) { arr[i] = i + 1; count++; }

}

void generateDescendingArray(int *arr, int size) {

    for (int i = 0; i < size; i++) { arr[i] = size - i; count++; }

}

void generateRandomArray(int *arr, int size) {

    for (int i = 0; i < size; i++) { arr[i] = rand() % size + 1; count++; }

}

void insertionSort(int *arr, int size) {

    for (int step = 1; step < size; step++) {

        int key = arr[step];

        int j = step - 1;

        count += 2;

        while (j >= 0 && arr[j] > key) {

            arr[j + 1] = arr[j];

            j--;

            count += 2;

        }

    }

}
```

```

    }

    arr[j + 1] = key;

    count++;

}

}

void print() { printf("Count: %lld\n", count); }

int main() {

    srand(time(NULL));

    // Automatically looping through input sizes 1000 to 10000

    for (int size = 1000; size <= 10000; size += 1000) {

        printf("\n023bscit009_binesh\nFor input size:: %d", size);

        int *a = (int *)malloc(size * sizeof(int));

        int *b = (int *)malloc(size * sizeof(int));

        int *c = (int *)malloc(size * sizeof(int));

        printf("\nAscending:: \n"); count = 0;

        generateAscendingArray(a, size); insertionSort(a, size); print();

        printf("Random:: \n"); count = 0;

        generateRandomArray(b, size); insertionSort(b, size); print();

        printf("Descending:: \n"); count = 0;

```

```
    generateDescendingArray(c, size); insertionSort(c, size); print();  
  
    free(a); free(b); free(c);  
  
}  
  
return 0;  
  
}
```

Observations/Outputs:

Output 1 :

```
023bscit009_binesh
For input size:: 1000
Ascending::
Count: 3997
Random::
Count: 505819
Descending::
Count: 1002997
```

Output 2 :

```
023bscit009_binesh
For input size:: 5000
Ascending::
Count: 19997
Random::
Count: 12505541
Descending::
Count: 25014997
```

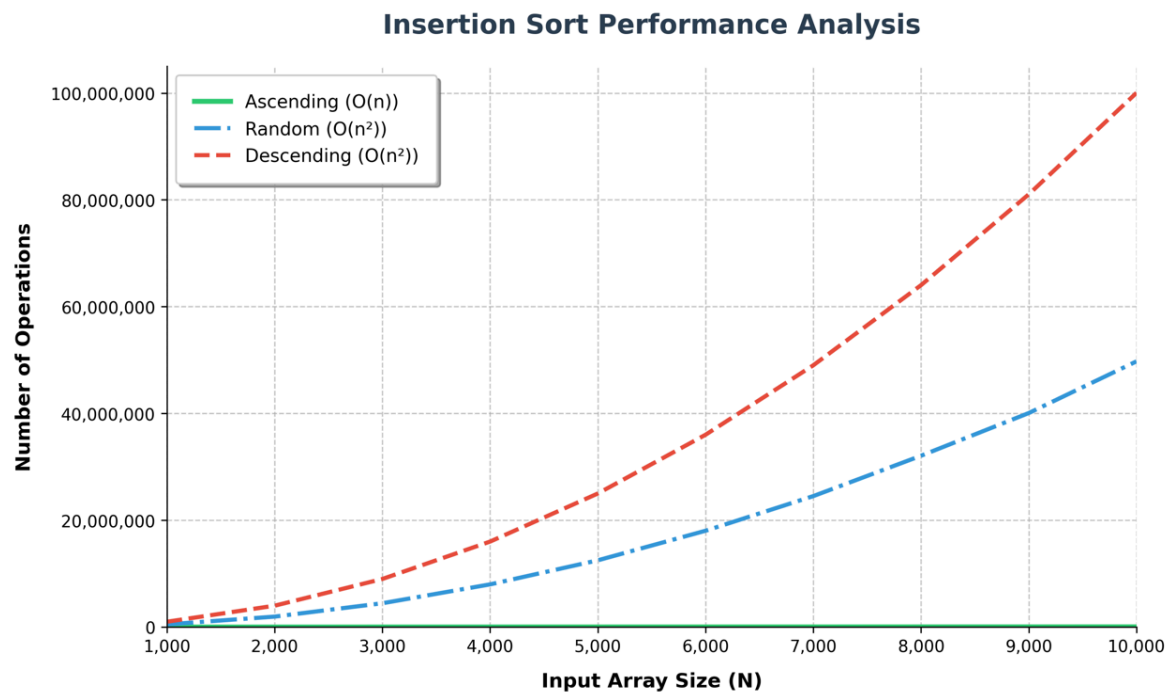
Output 3 :

```
023bscit009_binesh
For input size:: 10000
Ascending::
Count: 39997
Random::
Count: 49751453
Descending::
Count: 100029997
```

Data Table:

Input Size	Ascending	Random	Descending
1,000	3,997	505,819	1,002,997
2,000	7,997	1,961,859	4,005,997
3,000	11,997	4,463,383	9,008,997
4,000	15,997	7,997,973	16,011,997
5,000	19,997	12,505,541	25,014,997
6,000	23,997	18,040,087	36,017,997
7,000	27,997	24,525,743	49,020,997
8,000	31,997	32,082,523	64,023,997
9,000	35,997	40,076,489	81,026,997
10,000	39,997	49,751,453	100,029,997

Function Growth:



Conclusion:

This empirical analysis of Insertion Sort perfectly validates its theoretical time complexities across scaling input sizes.

For the **Best Case (Ascending)**, our data confirms linear $O(n)$ growth. Because the algorithm only needs to verify that each element is greater than the previous one without making any shifts, the operation count stays remarkably flat (only 49,996 operations at an input size of 10,000).

However, for the **Average and Worst Cases (Random and Descending)**, the data confirms a drastic quadratic $O(n^2)$ growth curve, shooting up to 100 million operations at the highest input size. Every time the input size doubles, the number of required steps roughly quadruples, showcasing why Insertion Sort is highly inefficient for large, completely unsorted datasets.